

Fair Drop — Fairness Model & Metrics

Every metric here is computed **by the server from authoritative drop state + post-hoc uploaded labels**, and independently by the simulator as a cross-check. A metric with no data is displayed as "—". Nothing is estimated or filled in.

1. Definition of fairness used

Within the eligible pool, every logical participant has the same probability of being allocated, independent of when, how fast, or how often they sent requests. End-to-end, participants judged abusive (restricted, failed challenge, duplicate identity) may have *lower* probability. They should never have higher.

This is **equal opportunity per logical participant**. It is not equal outcome per human, because a human controlling K accounts is K participants (see Sybil in the threat model).

2. Populations and notation

For a run with groups $g \in \{\text{human}, \text{bot}\}$ (and per profile):

Symbol	Meaning
R_g	logical participants of group g that attempted to register (ground truth from labels)
E_g	of those, ended ELIGIBLE (FAIR). In NAIVE, $E_g = R_g$ (no eligibility stage)
A_g	allocated : initial lottery winners + promotions (FAIR), or obtained a hold (NAIVE)
W_g	won : ended with a CONFIRMED ticket
q_i	total requests sent by participant i (all endpoints, incl. amplification and retries, counting edge-blocked ones the simulator observed)
N	total inventory (500)

Probabilities are reported with **Wilson 95 % confidence intervals**.

3. Core metrics

3.1 Group allocation rates (required)

- **Human Allocation Rate** $HAR = A_{human} / E_{human}$
- **Bot Allocation Rate** $BAR = A_{bot} / E_{bot}$
- End-to-end variants (include filtering effect): $HAR_{e2e} = A_{human} / R_{human}$, $BAR_{e2e} = A_{bot} / R_{bot}$
- **Bot Advantage Ratio** $BAdv = BAR_{e2e} / HAR_{e2e}$. 1 = parity, < 1 = bots disadvantaged, > 1 = bots advantaged.
- **Bot ticket share vs population share:** $TS_{bot} = W_{bot} / (W_{human} + W_{bot})$ compared with $PS_{bot} = R_{bot} / R$.

Expectation under an ideal FAIR lottery: $HAR = BAR = \min(1, N/E)$ up to sampling noise. Any remaining difference between BAR_{e2e} and HAR_{e2e} comes from the filtering layers (Mechanisms 2/3).

3.2 Request-Volume Advantage (directly measures "does sending more help?")

Two views, both reported:

(a) Cross-sectional RVA. Bucket participants by q_i into $\log_2$ buckets $B_k = [2^k, 2^{k+1})$. Win rate per bucket $w_k = A_k / R_k$.

$$RVA = w_{high} / w_{base}$$

w_{base} = allocation rate of participants with $q \leq 4$

w_{high} = allocation rate of participants with $q \geq 64$

Computed twice:

- $RVA_{eligible}$ (eligible participants only) isolates **allocation neutrality**. Ideal = 1.0.
- RVA_{e2e} (all registered) shows the **whole system**. Ideal ≤ 1.0 .

Also reported: **volume elasticity β** , the coefficient of $\log_{10}(q_i)$ in a participant-level logistic regression $allocated \sim \beta_0 + \beta \cdot \log_{10}(q)$, with 95 % CI. $\beta \approx 0$ means volume has no effect. For example, $\beta = 0.5$ means each 10× more requests multiplies the odds of allocation by $e^{0.5} \approx 1.65$.

(b) Interventional Volume Sensitivity (the primary claim). Run the same population and seed, changing only the bot cohort's `request_amplification` from 1× to A× (e.g. 1×, 10×, 100×, 1000×):

$$VS(A) = BAR_{e2e}(amp = A) / BAR_{e2e}(amp = 1)$$

Significance is a two-proportion z-test on the allocation counts. This is a controlled experiment, not a correlation, so it is the headline answer to "*does drastically increasing bot request volume significantly increase allocation probability?*". Expected outcome: FAIR vs ≈ 1 (or < 1 as more volume triggers controls), NAIVE vs > 1 if the declared capacity model sheds load.

3.3 Speed advantage

- **Arrival Advantage Ratio** $AAR = w(\text{earliest } 10\% \text{ by first-register time}) / w(\text{latest } 10\%)$, among eligible humans. Ideal 1.0.
- **Jain's fairness index over arrival deciles (humans):** $J = (\sum x_d)^2 / (10 \cdot \sum x_d^2)$, where x_d = allocation rate of human arrival decile d ($d = 1..10$). Range is $[0.1, 1]$. 1 means all arrival deciles have the same allocation rate. **Interpretation:** how far speed among legitimate users affects outcome. Because finite samples make $J < 1$ even for a perfect lottery, the server also reports the **null band**: the 2.5–97.5 percentile of J over 1,000 random relabelings of the same winner count. "Fair" = J inside the null band. Jain is deliberately **not** computed across human vs bot. Excluding abusers is desirable there and would wrongly reduce the index.

3.4 Sybil / multi-account exposure (honest residual)

- **Operator win rate** $OWR = \text{operators with } \geq 1 \text{ win} / \text{operators}$ and **tickets per operator** (mean, max) for bot operators that control many accounts (`operator_ref` from labels).
- Under FAIR, an operator with K eligible accounts expects $\approx K \cdot N/E$ wins. This is reported, not hidden.

4. Integrity metrics (must all be 0)

Metric	Definition
oversells	$\max(0, \text{held} + \text{confirmed} - N)$ at any checked point (max over the run) + count of confirmations beyond N
duplicate_allocations	participants holding > 1 ticket/reservation in HELD/CONFIRMED + accounts with > 1 participant in the drop
duplicate_ticket_holders	tickets referenced by > 1 active reservation
inventory_drift	`max
invalid_reservations	reservations for non-allocated participants, or confirmed after <code>expires_at</code>
lottery_entry_violation	$\text{lottery_entries} - \text{eligible_count}$ (must be 0: one entry per eligible participant)

5. Abuse-response metrics

`blocked_edge` (L0), `rate_limited` (L1/L2), `deduplicated` (L3 + L4 collapses), `throttled`, `challenged`, `challenge_passed`, `challenge_failed`, `restricted`, `ineligible_by_reason{...}`. Once labels arrive, each is also split by true label, giving **precision/recall of the risk engine** at CHALLENGE and RESTRICT, and **human false-positive rate** = humans challenged or restricted / humans. These are clearly marked *model-dependent* in logical mode (A5).

6. Performance metrics

Real HTTP mode: client-measured P50/P95/P99 latency, requests/s, error rate by code. Server-side DO processing time histogram (both modes). Logical mode reports **server processing latency only** and labels it as such.

7. ExperimentResults shape

```
ts
{
  summary: { experiment_id, comparison_id, event_id, drop_id, mode, sim_mode, scenario,
            total_participants, humans, bots, tickets, total_requests, duration_ms,
seeds:{population, allocation_commitment} },
  abuse:    { blocked_edge, rate_limited, deduplicated, throttled, challenged, restricted,
ineligible_by_reason, by_label:{...}, risk_precision_recall },
  integrity: { oversells, duplicate_allocations, duplicate_ticket_holders, inventory_drift,
invalid_reservations, lottery_entry_violation },
  fairness: { HAR, BAR, HAR_e2e, BAR_e2e, BAdv, TS_bot, PS_bot, RVA_eligible, RVA_e2e,
beta:{value, ci}, AAR, jain:{value, null_band}, OWR, per_profile:[...] , all with ci },
  performance:{ p50, p95, p99, rps_peak, error_rate, server_proc_p50/p95/p99 },
  budget:     { rows_written_do, rows_written_d1, worker_requests },
  provenance:{ server_version, simulator_version, risk_engine_version,
allocation_algorithm, audit_record_id }
}
```

8. What the metrics do *not* claim

- They don't prove a real-world bot cannot pass detection. Detection metrics depend on the synthetic behavior model.
- They don't measure fairness *per human being* when people run multiple accounts.
- NAIVE-mode unfairness depends on the declared capacity model, which is shown in the config next to the results.